



Netflix Clone

A Netflix-style streaming UI on live TMDB data

Part 2 of 2 — **Codebase & File Guide**

What every important file does, and how the pieces fit together.

Prepared 14 July 2026 · Author: Bhanu Prakash

o • The folder map

A monorepo with an npm-orchestrated `client/` (React + Vite) and `server/` (Express + MySQL). The server is a thin backend-for-frontend that owns auth + the watchlist and proxies all movie content from TMDB.

```
Netflix Clone/
├─ package.json          # root: dev / db:setup / install:all (concurrently)
├─ server/               # Express + MySQL BFF (port 5000)
│   └─ src/
│       ├─ index.ts      # app entry: middleware + routes
│       ├─ config/db.ts  # mysql2 connection pool
│       ├─ db/schema.sql # users + watchlist DDL
│       ├─ db/setup.ts   # applies schema.sql
│       ├─ middleware/   # requireAuth, error handler
│       ├─ routes/       # auth, movies, watchlist
│       └─ services/tmdb.ts # TMDB fetch wrapper (key stays here)
└─ client/              # React + Vite SPA (port 5173)
    └─ src/
        ├─ main.tsx / App.tsx # entry + router + provider tree
        ├─ api/               # axios client + typed API layer
        ├─ context/          # AuthContext + WatchlistContext
        ├─ components/       # Navbar, Row, MovieCard, MovieModal...
        └─ pages/            # Home, Search, MyList, Login, Register
```

Data flow in one sentence: a React page → the typed API layer (`api/index.ts`) → an Axios instance with a Bearer-token interceptor → Vite proxies `/api` → Express → either the `mysql2` pool (auth + watchlist) or TMDB (movie data). Posters and trailers load directly in the browser from the TMDB / YouTube CDNs; only the data API is proxied.

1 • Server (server/src)

ENTRY POINT

server/src/index.ts

The Express app: `cors` + JSON middleware, mounts `/api/auth`, `/api/movies`, `/api/watchlist`, and `/api/health`, wires the central error handler, and listens on port 5000.

DATABASE POOL

server/src/config/db.ts

Exports a shared `mysql2/promise` connection **pool** (`connectionLimit: 10`). Every query borrows from the pool and uses parameterised statements.

SCHEMA & SETUP

server/src/db/schema.sql • db/setup.ts

`schema.sql` is the DDL that creates the `netflix_clone` database and the `users` and `watchlist` tables (with the composite `UNIQUE` key + `ON DELETE CASCADE`). `setup.ts` reads and executes it (`npm run db:setup`).

AUTH

server/src/middleware/auth.ts • routes/auth.ts

`routes/auth.ts` handles `POST /register`, `POST /login` (bcrypt verify → sign a 7-day JWT), and the protected `GET /me`. `middleware/auth.ts` exposes `requireAuth`, which verifies the Bearer token and sets `req.userId`.

★ MOVIES (THE TMDB PROXY)

server/src/routes/movies.ts • services/tmdb.ts

`routes/movies.ts` exposes `/banner` (a random trending title), `/categories` (eight endpoints fetched in parallel via `Promise.all`), `/search` (multi-search), and `/:type/:id` (details + best YouTube trailer via `append_to_response=videos`). A `normalize()` mapper converts TMDB's snake_case into the client's camelCase `Movie` shape. `services/tmdb.ts` is the fetch wrapper that holds the key and sends it as a Bearer header (v4 token) or `api_key` query param (v3 key).

WATCHLIST

server/src/routes/watchlist.ts

Protected CRUD: `GET /` (the user's list), `POST /` (idempotent upsert via `ON DUPLICATE KEY UPDATE`), `DELETE /:tmdbId`. `middleware/errorHandler.ts` defines an `HttpError` class + the Express error handler used across routes.

2 • Client (client/src)

FILE	PURPOSE
<code>main.tsx</code>	React entry — mounts <code><App/></code> in StrictMode.
<code>App.tsx</code>	Router + provider tree (<code>AuthProvider</code> > <code>WatchlistProvider</code>), the route table, <code>ProtectedRoute</code> , and <code>AppLayout</code> (Navbar + <code><Outlet/></code>).
<code>vite.config.ts</code>	Vite config: React + Tailwind plugins and the <code>/api</code> → <code>:5000</code> dev proxy.
<code>api/client.ts</code>	The Axios instance (<code>baseUrl: '/api'</code>) + the request interceptor that attaches the Bearer token from <code>localStorage</code> ; <code>TOKEN_KEY</code> , error-message helper.
<code>api/index.ts</code>	The typed API layer — one function per endpoint (auth, banner, categories, details, search, watchlist).
<code>types/index.ts</code>	Shared TypeScript interfaces (<code>User</code> , <code>Movie</code> , <code>MovieDetails</code> , <code>Category</code> , <code>WatchlistItem</code>).
<code>lib/image.ts</code>	<code>tmdbImage()</code> — builds TMDB image URLs by size.
<code>context/AuthContext.tsx</code>	Session/user state; <code>login</code> / <code>register</code> / <code>logout</code> ; <code>useAuth</code> .
<code>context/WatchlistContext.tsx</code>	Watchlist state; <code>isSaved</code> / <code>toggle</code> / <code>remove</code> with optimistic updates; refetches when the user changes.

Components

FILE	PURPOSE
<code>components/Navbar.tsx</code>	Fixed nav: logo, links, expanding search input, user name, sign-out; transparent → black on scroll.
<code>components/HeroBanner.tsx</code>	Full-bleed hero backdrop with Play + My List actions.
<code>components/Row.tsx</code>	A horizontal scroll row of <code>MovieCard</code> s with arrow buttons.
<code>components/MovieCard.tsx</code>	Poster thumbnail button with hover overlay / fallback tile.
<code>components/MovieModal.tsx</code>	Detail modal — YouTube trailer, metadata, % match, genres, My List toggle; Esc/backdrop close.
<code>components/ProtectedRoute.tsx</code>	Auth gate — spinner while loading, redirect to <code>/login</code> if no user.

Pages

FILE	PURPOSE
<code>pages/Home.tsx</code>	Browse page — fetches banner + categories, renders hero + eight rows + the modal.
<code>pages/Search.tsx</code>	Debounced (400 ms) multi-search results grid.

<code>pages/MyList.tsx</code>	Watchlist grid with remove buttons.
<code>pages/Login.tsx</code> · <code>Register.tsx</code>	Sign-in / sign-up forms.

3 • Commands

COMMAND	WHAT IT DOES
<code>npm run install:all</code>	Installs root + server + client dependencies.
<code>npm run db:setup</code>	Creates the <code>netflix_clone</code> MySQL database + tables from <code>schema.sql</code> .
<code>npm run dev</code>	Runs server (<code>:5000</code>) + client (<code>:5173</code>) together via <code>concurrently</code> .
<code>npm run build</code> / <code>start</code>	Type-check + build both; run the compiled server.

Prerequisites: a running local **MySQL** whose credentials match `server/.env`, internet access for TMDB, and a TMDB API key (already wired). There is **no seed / demo user** — the tables start empty, so register an account to get in.